Everything's Computer! A New Paradigm Shift:

A Comprehensive Analysis of Distributed, Intelligent Operating System Architectures for the Era of Networked Compute. (BE-AI Research)

1. Executive Summary: The Dawn of the Adaptive Kernel

Modern computing is at a critical inflection point, driven by the explosive proliferation of edge devices, AI workloads, and vast cloud infrastructures. The traditional operating system, with its roots in monolithic kernels designed for isolated, single-processor machines, is becoming an architectural anachronism. This report, building upon the foundational work presented in *Everything's Computer!*, argues for a radical paradigm shift: the redefinition of the "computer" from a static, single machine to a distributed, intelligent system.[1]

The core thesis is that the kernel must evolve from a static resource manager into an adaptive, AI-integrated platform. To this end, two distinct, yet potentially complementary, kernel architectures are proposed and examined in detail. The first, the **Neurosymbolic Kernel**, envisions the operating system as a computational graph, treating loadable kernel modules (LKMs) as first-class computational units. It integrates AI-driven scheduling and probabilistic reasoning to adapt dynamically to diverse workloads.[1] The second, the

Hypervisor-based Kernel, represents a minimalist approach, delegating nearly all high-level functionality to virtual machines (VMs) and focusing the kernel's role on secure, high-assurance hardware abstraction and inter-VM isolation.[1]

This report provides a comprehensive analysis of these two architectural models. It synthesizes historical design principles, such as modularity and compositionality, with modern

trends in high-performance computing (HPC) and the Artificial Intelligence of Things (AIoT) to validate the underlying concepts.[1] The analysis delves into the technical and practical considerations of each model, including their respective security postures, performance profiles, and ideal use cases. By doing so, the report elevates the initial proposition into a detailed framework for the next generation of operating system design, offering a clear blueprint for transforming computing in the era of networked resources.

2. Introduction: The Modern Computing Imperative

The foundational architecture of modern operating systems, largely centered around the monolithic kernel model, has remained fundamentally unchanged for decades.[1] While incremental improvements—such as multitasking, memory protection, and network stacks—have extended the life of this model, they have not altered its core design, which assumes a machine-centric rather than a network-centric worldview.[1] This legacy architecture presents a significant obstacle to the seamless and efficient orchestration of today's heterogeneous compute landscape, which includes everything from resource-constrained edge devices and specialized accelerators (GPUs, TPUs) to vast cloud clusters.[1]

The central challenge is a profound architectural mismatch. Modern applications require a system that can fluidly allocate computational tasks and data across a geographically and topologically diverse network, often in real-time. A monolithic kernel, with its tightly coupled services and large attack surface, is ill-equipped to meet these demands for low-latency, scalable, and secure operations.[5] The limitations manifest in several key areas, including high overhead from inter-process communication, vulnerabilities that can compromise the entire system, and an inability to dynamically reconfigure the system to meet changing workload requirements without recompilation or a reboot.[1]

This report, therefore, takes the foundational concepts of the *Everything's Computer!* paper and expands them into a comprehensive, expert-level analysis. The scope is to meticulously examine the technical underpinnings of the two proposed kernel architectures—the Neurosymbolic Kernel and the Hypervisor-based Kernel—and to situate them within a broader context of historical and contemporary computer science research. The analysis extends to a direct comparison of the two models, highlighting their respective strengths and weaknesses and identifying how they might be combined into a single, cohesive framework.

The analysis demonstrates that the move from a device-centric to a network-centric view of computing is a powerful parallel to the industry's broader shift toward distributed systems and cloud-native development.[7] By embracing a model where computing, storage, and networking are treated as a unified resource pool, the proposed architectures align with the

principles that have enabled the scalability and flexibility of modern cloud platforms.[9] Furthermore, they lay the groundwork for a future dominated by AIoT, where real-time, on-device intelligence is not just a feature, but a fundamental capability of the system itself.[3]

3. Foundational Principles: A Synthesis of Historical Design and Modern Requirements

The proposed kernel architectures do not emerge from a vacuum; they are grounded in a synthesis of four fundamental design principles drawn from the history of operating systems.[1] Their continued relevance lies in a new interpretation that accommodates networked, heterogeneous resources, thereby providing a rigorous foundation for future designs.

3.1 Modularity and Compositionality

The principle of **modularity**, the practice of breaking a system into interchangeable components, is a direct response to the inherent complexity and rigidity of monolithic kernels.[1] By contrast, microkernels, a foundational alternative, aim to place only a minimal set of services in a privileged kernel space, with all other services—such as file systems and device drivers—running as separate processes in user space.[6] This approach, while potentially incurring higher inter-process communication (IPC) overhead, significantly enhances system stability and security. A bug in a device driver, for example, cannot crash the entire system or silently modify the kernel's core state, as it is isolated in its own memory space.[6] The proposed architectures, with their emphasis on loadable modules and virtualized environments, represent a culmination of this historical trend toward greater modularity and a smaller, more secure core.

Compositionality, the principle of building complex behavior from simple, well-defined parts, is elevated to a formal science in the proposed Neurosymbolic kernel architecture.[1] By modeling computational elements as mathematical objects and their transformations as morphisms—a concept borrowed from

Category Theory—the kernel can compose services in a mathematically rigorous way.[13] This formal approach allows the system to provably reason about dependencies and resource usage, which is crucial for reducing errors and enhancing security in complex, distributed environments.[1] This is in stark contrast to traditional software engineering, where complex system behavior is often emergent and difficult to verify.

3.2 Resource-Exact-Once Semantics and Separation of Concerns

The principle of **Resource-Exact-Once Semantics** ensures that each resource is used exactly once, preventing leaks and race conditions.[1] This is not merely a best practice; it is a critical countermeasure to the fundamental challenges of distributed systems. In a networked environment, components can fail independently, and network operations can result in an "UNKNOWN" state, making it impossible to know if a request succeeded or failed.[15] By enforcing strict resource usage policies, the kernel provides a deterministic foundation for scheduling and resource allocation, mitigating the non-determinism and independent failures that are the most impactful issues in distributed systems.[15]

Separation of Concerns, the design practice of distinguishing layers such as hardware abstraction and process management, is the driving force behind the Hypervisor-based kernel.[1] This approach aligns perfectly with the

Separation Kernel philosophy, a bare-metal hypervisor design that is even smaller and more secure than traditional Type 1 hypervisors.[18] A separation kernel contains no device drivers, no user model, and no dynamic memory; its sole function is to provide strict isolation between software partitions, ensuring that information can only flow along explicitly defined paths.[18] This architectural pattern, originally developed for high-security government and safety-critical avionics applications, provides a proven blueprint for building a clean, auditable, and secure kernel core.[18] This alignment reveals that the proposed Hypervisor-based kernel is not a completely new idea, but a modern application of a long-established, high-assurance design pattern to the problem of distributed computing. It is a powerful validation of the proposed architecture's security and reliability, as it is rooted in a philosophy that prioritizes a minimal Trusted Computing Base (TCB) to limit vulnerabilities and enhance formal verification.[18]

4. The Neurosymbolic Kernel: An Adaptive AI-Integrated Core

The Neurosymbolic Kernel is an architectural model that re-envisions the kernel as an adaptive, intelligent platform rather than a static resource manager.[1] At its core, the system is a computational graph where nodes represent either traditional system services or AI inference units.[1] Each node is implemented as a loadable module that can be dynamically swapped in or out, allowing the OS to adapt its structure to changing workload profiles without a system reboot.[1]

4.1 Formal and Probabilistic Foundations

A defining feature of this architecture is its grounding in formal methods. By leveraging **Category Theory and Homotopy Type Theory**, the kernel can model its components as objects and their interactions as morphisms.[1] This provides a mathematically rigorous framework for defining and verifying the composition of services. For instance, the system can formally reason about dependencies and resource usage, which significantly reduces the potential for errors and enhances system security by guaranteeing the correctness of module interactions.[1]

This formal structure is complemented by a **probabilistic reasoning layer**. This layer incorporates probabilistic models that allow the kernel to evaluate uncertainty and make dynamic, data-driven decisions.[1] For example, if the confidence level of an AI inference module drops below a certain threshold, the kernel can probabilistically determine the best course of action, such as offloading the task to a more reliable node in the network.[1] This departs from traditional, deterministic systems by enabling a fluid, responsive approach to resource management. The concept is not without precedent; probabilistic scheduling algorithms, such as

Lottery Scheduling, have long been used in operating systems.[22] In a lottery scheduler, processes are assigned a number of "tickets," and the scheduler randomly draws a ticket to select the next process, with the number of tickets determining a process's chance of being selected.[22] This probabilistic approach can approximate other scheduling algorithms and, most importantly, guarantees that every process has a non-zero probability of being selected, thus preventing starvation.[22]

4.2 AI-Integrated Scheduling

The Neurosymbolic Kernel replaces traditional priority-based schedulers with adaptive, learning-based policies.[1] These policies consider a wide array of factors in real-time, including workload characteristics, hardware capabilities, and network latency.[1] The scheduler itself is a loadable module that can update its policy based on feedback from ongoing tasks.[1]

The Neurosymbolic scheduler can be understood by drawing parallels to AI-driven tools in other domains. Consider how an AI-powered project management system uses real-time data to optimize project timelines and resource allocation.[23] These systems process vast amounts of data to forecast a project's timeline and resources, making continuous adjustments as new information becomes available.[23] Similarly, an AI-integrated kernel would ingest real-time data from across the distributed system—from CPU load on a cloud server to the temperature of an edge sensor—and use machine learning algorithms to predict future resource needs. It would then dynamically adjust its scheduling policies to meet those needs. For example, if it learns that a particular workload is consistently delayed by network latency, it could re-prioritize local compute to ensure the task meets its deadline. This transformation changes the scheduler from a rigid, rule-based system into a fluid, predictive one. The more it runs, the better it becomes at optimizing resource utilization and performance, much like an AI calendar that learns to avoid rescheduling meetings on Friday afternoons.[24]

5. The Hypervisor-Based Kernel: A Minimalist and Secure Foundation

The Hypervisor-based Kernel offers an alternative architectural vision: a minimalist, high-assurance foundation designed for security and flexibility.[1] This model delegates almost all functionality to guest operating systems and application-level packages running in VMs.

5.1 Core Architecture and I/O Mediation

The architecture is composed of three distinct layers:

1. **Hardware Abstraction Layer (HAL):** A small, statically-linked binary that is the only component to directly touch the CPU, memory, and peripheral buses.[1] Its sole purpose is to expose raw device registers and low-level I/O primitives to the hypervisor.[1]
2. **Hypervisor Interface:** A thin API that provides only the essential hypercall primitives for managing VMs, such as context switching, memory mapping, and device passthrough.[1] All higher-level services are intentionally left to the guests.[1]
3. **Guest OS / Web-Only Packages:** These run inside the VMs and perform all application-level functions. They can be traditional OS kernels like Linux or Windows, or lightweight stacks like Docker containers or WASM runtimes.[1]

Because the kernel contains no device drivers or networking stack, the HAL must mediate shared I/O across multiple guests.[1] This mechanism draws a direct parallel to the early IBM System/360/370 networking model, where a dedicated I/O domain serviced virtualized spool files, allowing guests to interact with I/O as if it were local.[1] In this modern interpretation, a centralized I/O domain—a dedicated VM or a lightweight hypervisor thread—owns the physical network interfaces and forwards packets from guests to their destination.[1] This centralization allows the HAL to enforce fairness, Quality of Service (QoS), and security policies across all guests, providing a clean, auditable, and secure approach to network management.[1]

The decision to adopt a design with historical precedent is a powerful one. It indicates that the proposed solution is not a theoretical novelty but a modern interpretation of a proven, scalable, and secure architecture. The System/360 model, with its dedicated I/O channels and modular design, demonstrated that a centralized I/O mediation layer could efficiently handle resource-sharing in a multi-tenant environment.[25] The Hypervisor-based kernel leverages this same principle to solve the shared I/O problem in a distributed, virtualized context.

5.2 Security and Performance

The Hypervisor-based kernel aligns with the separation kernel philosophy by maintaining a minimal, high-assurance core that is easier to audit, formalize, and secure.[1] By isolating services in separate VMs, the system can contain faults and prevent their propagation, providing strong fault containment and a reduced attack surface.[6] The security benefits of such a minimalist design are well-documented, with some separation kernels, such as seL4,

even having been formally verified for correctness.[12]

While hypervisors are often associated with performance overhead, especially due to context switching between the hypervisor and its guests, this can be mitigated with modern techniques.[27] Benchmarks show that with proper optimization, such as

paravirtualized drivers that notify the hypervisor only when necessary, the CPU overhead can be kept as low as 1 to 5 percent for typical workloads.[27] Techniques like

zero-copy can further reduce I/O latency for critical traffic.[1] By eliminating the intermediary host OS, bare-metal hypervisors can also achieve superior performance and lower latency, making them ideal for high-performance workloads such as AI and real-time processing.[29] The Hypervisor-based kernel, by design, leverages these advantages to provide a secure, low-overhead foundation.

6. Comparative Analysis: Neurosymbolic vs. Hypervisor-based

The two kernel architectures, while both designed to address the limitations of monolithic kernels, represent two distinct and powerful paths forward. A direct comparison reveals their unique trade-offs and complementary strengths.

Characteristic	Neurosymbolic Kernel	Hypervisor-based Kernel
Core Design Philosophy	AI-integrated, adaptive, treats kernel as a computational graph.	Minimalist, secure, delegates functionality to VMs.
Primary Security Mechanism	Formal verification via category theory, probabilistic reasoning.	Small, auditable Trusted Computing Base (TCB) and strict isolation between guests.
Extensibility Model	Dynamic loading and unloading of AI-oriented Loadable Kernel Modules (LKMs).	"Plug-and-play" guest OSes and web-only packages.
Performance Profile	Optimized for AI workloads; potential latency from probabilistic reasoning layer.	Minimal overhead (1-5% CPU) but with potential latency from context switching, mitigated by paravirtualization.
I/O & Networking	Distributed compute as a unified resource pool.	Centralized I/O domain managed by the HAL.
Ideal Use Cases	Edge-AI, real-time analytics, distributed HPC, and systems requiring unified intelligence.	Multi-tenant cloud computing, safety-critical systems, secure workstations, and environments requiring strong isolation.

The Neurosymbolic kernel is fundamentally about extending the kernel's capabilities, embedding a layer of intelligence within the trusted core to enable probabilistic and adaptive behavior.[1] Its strength lies in its ability to dynamically reconfigure itself and intelligently orchestrate resources for complex, distributed tasks. In contrast, the Hypervisor-based kernel is a lesson in minimalism; its strength is not in what it does, but in what it

doesn't do.[1] By delegating functionality and focusing on isolation, it offers a secure, high-assurance foundation that is ideal for multi-tenant and safety-critical environments.[18]

A compelling observation is that these two architectures are not mutually exclusive. The Hypervisor-based kernel is designed to host any OS kernel that implements its hypercall ABI, including legacy kernels and modern ones.[1] This suggests a profound potential for a hybrid architecture: the Neurosymbolic kernel could be implemented as a specialized, highly-privileged "service VM" running on top of the Hypervisor-based kernel. This would combine the strengths of both models, leveraging the unassailable security and formal verification potential of the minimalist hypervisor while gaining the adaptive, AI-driven intelligence of the Neurosymbolic design. The Hypervisor-based kernel would provide the immutable, secure foundation, and the Neurosymbolic kernel would provide the intelligent, flexible layer for managing distributed resources.

7. The Logical Workstation: Implications for a Distributed Future

In this new paradigm, the computer ceases to be a single physical machine. It becomes a **logical node**, an abstraction that aggregates computational, storage, and networking resources across a distributed system.[1] Users interact with this logical entity as if it were a single machine, while the kernel transparently manages the underlying heterogeneity and distributed nature of the system.[1]

7.1 Addressing Distributed System Challenges

This vision directly addresses the most significant challenges of distributed computing. Problems like independent failures, resource contention, and data consistency, which are notoriously difficult to debug and manage in traditional systems, are handled at the kernel

level.[15]

- **Fault Tolerance:** The proposed architectures' reliance on modular components and isolated VMs provides inherent fault tolerance. If one module or VM fails, the rest of the system can continue operating, with the kernel potentially restarting the failed component.[6] The probabilistic reasoning layer of the Neurosymbolic kernel further enhances this by enabling the system to evaluate uncertainty and make decisions that account for potential failures.[1]
- **Resource Management:** The concept of a unified resource pool, where devices like edge sensors and cloud VMs are part of a single address space, allows for dynamic and intelligent resource allocation.[1] The kernel can enforce fairness and Quality of Service (QoS) across a network of heterogeneous devices, mitigating bottlenecks and preventing the starvation of lower-priority tasks.[1]
- **Data Locality & Latency:** The Neurosymbolic kernel can intelligently place workloads on the most suitable device (e.g., executing a latency-sensitive AI task on an edge sensor) to reduce network latency and preserve data locality.[1] This is critical for real-time applications that cannot tolerate the delays of a round trip to a centralized cloud.[31]

7.2 Real-World Parallels and Future Trajectories

The proposed **logical workstation** has powerful parallels in existing and emerging technological domains.

- **High-Performance Computing (HPC):** The unified resource pool model generalizes the principles of HPC clusters, which aggregate the power of multiple computers to solve complex problems at extremely high speeds.[2] Just as HPC uses a centralized scheduler to manage a parallel workload across thousands of nodes, the proposed kernels would use an adaptive scheduler to manage resources across a distributed network.[1] High-speed, low-latency interconnects, such as Remote Direct Memory Access (RDMA), are a cornerstone of modern HPC and would be essential to the logical workstation model, enabling seamless communication between nodes without involving the operating system.[32]
- **Artificial Intelligence of Things (AIoT):** The Neurosymbolic kernel's ability to treat LKMs as AI-oriented computation units is a foundational component of the AIoT architecture, which integrates AI directly into IoT devices at the network edge.[3] This model addresses the limitations of traditional cloud-based AI by enabling real-time analytics and decision-making on the device itself, reducing latency and bandwidth requirements.[3] The proposed system provides the perfect operating environment for this paradigm, allowing for distributed data collection, local processing, and intelligent,

real-time responses across a network of devices.[31]

- **Unikernels at the Edge:** For resource-constrained edge devices, the proposed loadable modules could be implemented as **unikernels**.[35] A unikernel is a highly specialized, single-purpose system that merges a single application with the essential parts of an operating system into a compact binary.[36] Their minimal footprint, fast boot times, and reduced attack surface make them an ideal choice for the lightweight, efficient, and secure components envisioned in the proposed architectures.[35]

8. Conclusion and Future Research Directions

The traditional operating system is reaching the limits of its utility in a world dominated by networked, heterogeneous compute. The two kernel architectures proposed in this report—the adaptive, AI-integrated Neurosymbolic Kernel and the minimalist, secure Hypervisor-based Kernel—offer two distinct yet viable paths for a fundamental architectural reset. The analysis demonstrates that these models are not theoretical abstractions but are deeply rooted in proven design principles and align with the most pressing trends in modern computing, from HPC and AIoT to the philosophy of separation kernels.

The ultimate value of this work lies in its potential for a powerful synthesis: a hybrid architecture where the Hypervisor-based kernel provides a secure, auditable, and immutable foundation, and the Neurosymbolic kernel runs as a privileged guest, providing the adaptive intelligence required for dynamic resource orchestration. This combined model would deliver the best of both worlds: uncompromised security with unparalleled flexibility and responsiveness.

To realize this vision, a detailed and systematic research and implementation roadmap is required. The following table outlines a four-phase plan to move this paradigm from a theoretical framework to a demonstrable, practical system.

Phase	Objectives	Milestones
1 – Prototype	Build a minimal kernel that supports AI-oriented loadable kernel modules (LKMs).	A functional LKM for a specific AI task (e.g., image classification); a basic message-passing API to allow the LKM to interact with the kernel; a resource registry to track available

		modules.
2 – Formal & Probabilistic Core	Implement the core mathematical abstractions and probabilistic reasoning layer.	Formal verification of a select kernel component using Category Theory principles; implementation of a basic probabilistic scheduler (e.g., a **Lottery Scheduler** or a **Hidden Markov Model**-based scheduler).
3 – Distributed Integration	Enable transparent remote execution and data locality across a network of heterogeneous devices.	Remote module loading capability; network-aware task placement logic; a unified address space across at least two physical nodes.
4 – Comparative Benchmarking	Benchmark the prototype against a traditional monolithic kernel on a range of edge-to-cloud workloads.	Quantifiable metrics for latency, throughput, energy consumption, and security; a detailed report comparing the proposed system's performance to a legacy kernel.

Works cited

1. Everythings Computer - A fresh perspective on OS design by using principles from the past.docx (BE-AI Research - Shayne Brown)
2. What is high performance computing (HPC) | Google Cloud, accessed September 17, 2025, https://cloud.google.com/discover/what-is-high-performance-computing
3. Artificial Intelligence of Things (AIoT) Guide - Synaptics, accessed September 17, 2025, https://www.synaptics.com/company/blog/artificial-intelligence-of-things-guide
4. Emerging Trends and Future Directions | Operating Systems Class Notes - Fiveable, accessed September 17, 2025, https://library.fiveable.me/operating-systems/unit-12
5. Kernel in Operating System - GeeksforGeeks, accessed September 17, 2025, https://www.geeksforgeeks.org/operating-systems/kernel-in-operating-system/
6. Microkernel Vs Monolithic Kernel Design Trade-Offs - IJFMR, accessed September 17, 2025, https://www.ijfmr.com/papers/2024/2/17896.pdf
7. Distributed computing - Wikipedia, accessed September 17, 2025, https://en.wikipedia.org/wiki/Distributed_computing
8. What is a distributed system? | Atlassian, accessed September 17, 2025, https://www.atlassian.com/microservices/microservices-architecture/distributed-architecture
9. The Impact of Cloud Technologies on Software Development - GeeksforGeeks, accessed September 17, 2025, https://www.geeksforgeeks.org/blogs/impact-of-cloud-technologies-on-software-development/
10. The Impact of Cloud Computing on Software Development - LeadGen App, accessed September 17, 2025, https://leadgenapp.io/the-impact-of-cloud-computing-on-software-development/
11. Microkernel - Wikipedia, accessed September 17, 2025, https://en.wikipedia.org/wiki/Microkernel
12. Is a monolithic kernel more secure than a microkernel for a small OS?, accessed September 17, 2025, https://security.stackexchange.com/questions/67102/is-a-monolithic-kernel-more-secure-than-a-microkernel-for-a-small-os
13. (PDF) Category-Theoretic Approach to Software Systems Design - ResearchGate, accessed September 17, 2025, https://www.researchgate.net/publication/286295228_Category-Theoretic_Approach_to_Software_Systems_Design
14. A Categorical Model for a Versioning File System - Instituto de Informática - UFRGS, accessed September 17, 2025, https://www.inf.ufrgs.br/~eslgastal/files/cmvfs.pdf
15. Challenges with distributed systems - AWS - Amazon.com, accessed September

17, 2025, https://aws.amazon.com/builders-library/challenges-with-distributed-systems/
16. Distributed Systems Challenges - by Veera Atukuri - Medium, accessed September 17, 2025, https://medium.com/@sairaju.atukuri123/distributed-systems-challenges-a19bb445fd22
17. Resource Sharing in Distributed System - GeeksforGeeks, accessed September 17, 2025, https://www.geeksforgeeks.org/operating-systems/resource-management-in-distributed-system/
18. What Is A Separation Kernel? - Lynx Software Technologies, accessed September 17, 2025, https://www.lynx.com/embedded-systems-learning-center/what-is-a-separation-kernel
19. Hypervisors solve all your software partitioning issues. Do they really? - Embedded, accessed September 17, 2025, https://www.embedded.com/hypervisors-solve-all-your-software-partitioning-issues-do-they-really/
20. Separation kernel - Wikipedia, accessed September 17, 2025, https://en.wikipedia.org/wiki/Separation_kernel
21. Separation Kernel as a Basis for certifiable Applications and Systems - SYSGO, accessed September 17, 2025, https://www.sysgo.com/professional-articles/separation-kernel-as-a-basis-for-certifiable-applications-and-systems
22. Lottery scheduling - Wikipedia, accessed September 17, 2025, https://en.wikipedia.org/wiki/Lottery_scheduling
23. (PDF) ADAPTIVE SCHEDULING: APPLYING AI AND MACHINE ..., accessed September 17, 2025, https://www.researchgate.net/publication/392703149_ADAPTIVE_SCHEDULING_APPLYING_AI_AND_MACHINE_LEARNING_TO_OPTIMIZE_PROJECT_TIMELINES_AND_RESOURCE
24. AI Schedulers: How to Automate Your Calendar with AI - LiveX AI Blogs, accessed September 17, 2025, https://www.livex.ai/blog/ai-schedulers-how-to-automate-your-calendar-with-ai
25. IBM System/360 architecture - Wikipedia, accessed September 17, 2025, https://en.wikipedia.org/wiki/IBM_System/360_architecture
26. OS/360 and successors - Wikipedia, accessed September 17, 2025, https://en.wikipedia.org/wiki/OS/360_and_successors
27. Virtualization's Impact on Performance Management - Dynatrace, accessed September 17, 2025, https://www.dynatrace.com/resources/ebooks/javabook/why-virtualization-has-impact-on-performance-management/
28. Is virtual machine slower than the underlying physical machine? - Server Fault, accessed September 17, 2025, https://serverfault.com/questions/135431/is-virtual-machine-slower-than-the-und

erlying-physical-machine
29. 8 Best Bare Metal Hypervisors for 2025 - Liquid Web, accessed September 17, 2025, https://www.liquidweb.com/blog/bare-metal-hypervisors/
30. Bare Metal Hypervisors: A Deep Dive For Everyone - RedSwitches, accessed September 17, 2025, https://www.redswitches.com/blog/bare-metal-hypervisors/
31. Distributed Systems and Internet of Things - GeeksforGeeks, accessed September 17, 2025, https://www.geeksforgeeks.org/system-design/distributed-systems-and-internet-of-things/
32. What Is High-Performance Computing (HPC)? - IBM, accessed September 17, 2025, https://www.ibm.com/think/topics/hpc
33. www.mdpi.com, accessed September 17, 2025, https://www.mdpi.com/2076-3417/15/9/4625#:~:text=The%20AIoT%20architecture%20addresses%20all,be%20implemented%20in%20every%20layer.
34. Understanding Distributed Edge Computing - Simply NUC, accessed September 17, 2025, https://simplynuc.eu/blog/distributed/
35. Exploring the Viability of Unikernels for ARM-powered Edge Computing - ResearchGate, accessed September 17, 2025, https://www.researchgate.net/publication/386454750_Exploring_the_Viability_of_Unikernels_for_ARM-powered_Edge_Computing
36. Enhancing Edge Computing with Unikernels in 6GNetworks - Research Explorer - The University of Manchester, accessed September 17, 2025, https://research.manchester.ac.uk/files/271642230/pimrc.pdf